Big Data Solution for Warsaw Public Transit delay prediction - Milestone 1 report

Big Data Analytics

Emil Łasocha Michał Szewczak Sebastian Trojan Tymoteusz Urban

October 19, 2026

1 Description

The goal of the project is to build a system that shows current delays in Warsaw's public transport, how they will change in the near future, and how these delays depend on weather and air conditions. The system will collect live positions of buses and trams, timetables, weather data and air quality data published by the city. By comparing the real position of a vehicle with its timetable we can calculate its delay. Then this information is combined with current weather and used both for live monitoring and for training a machine learning model that predicts delays.

Weather is an obvious candidate for a factor that affects delays, because rain, snow or ice make traffic slower. Air quality is less obvious, but we expect some correlation with delays, since heavy traffic both slows down buses and makes the air quality worse. One of the goals of the project is to check how strong these relations really are.

From the user perspective, the final product is a web dashboard which presents current delays, predicted delays and historical statistics in a simple form.

2 Project Deliverables and Users

2.1 Target users

We identified three main groups of users of the system:

- **Passenger** - a person who travels by bus or tram in Warsaw and wants to know if their line is on time and what delay to expect, especially in bad weather.
- **Dispatcher** - a person working for the transport operator who watches the whole network during the day and needs to react quickly when delays start to grow.
- **Transport planner / analyst** - a person who works on timetables and the transport network in the long term and wants to understand where and why delays appear.

2.2 Deliverables

After finishing the project, the users will get access to the following features:

- **Live delay monitoring** - current position and delay of vehicles, which can be filtered by line or stop, and a map of the network with delays marked on it. Next to the delay information, the dashboard also shows current weather and air quality measured by stations located in the city.

- **Delay prediction** - predicted delay of a given line at a given stop for the near future, calculated by a machine learning model working on streaming data and taking into account current weather and air quality. Based on the predictions, the dashboard also shows alerts with a list of lines and areas where the predicted delay is higher than a chosen threshold, so problems can be noticed beforehand.
- **Historical statistics and weather impact analysis** - average delays by line, stop, hour of the day, day of the week and weather conditions, calculated with batch processing on all collected data. This also includes a comparison of delays in different weather conditions (for example dry vs. rainy days) and a list of lines and stops which are most sensitive to bad weather.

2.3 Use cases

The use case diagram of the system is presented in Figure 1.

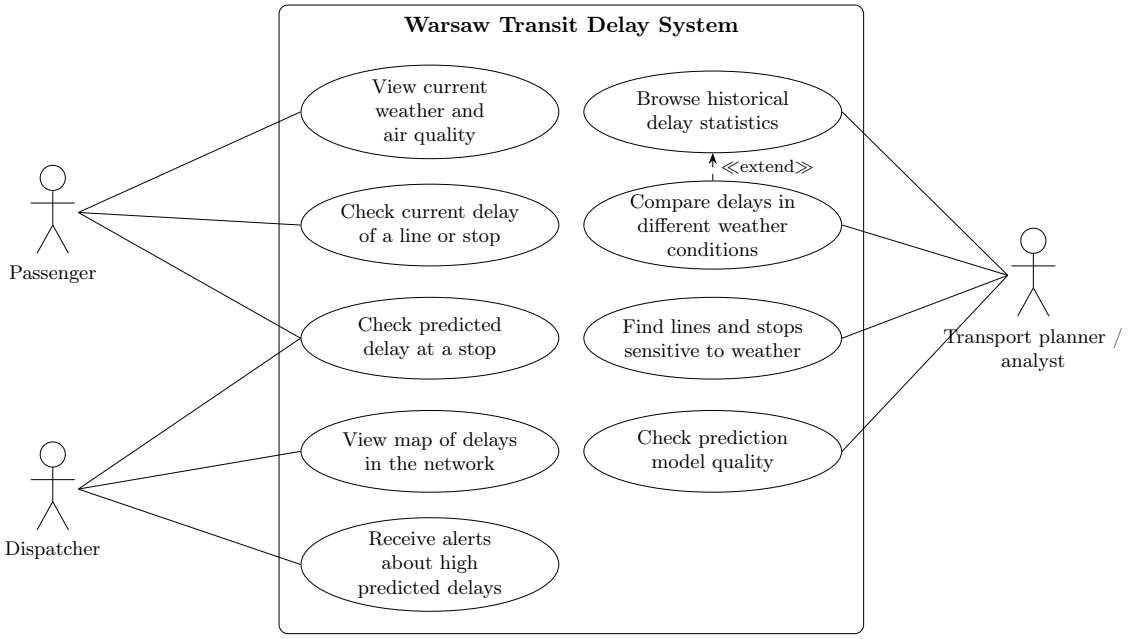


Figure 1: Use case diagram of the system

2.4 Business process diagrams

Figures 2 and 3 show the two core business processes of the system. The first one describes a passenger who checks the delay of their line before a trip. The second one describes a dispatcher who monitors delays in the whole network. Actions taken by the dispatcher after assessing the situation, such as sending a reserve vehicle, are outside the scope of our system, so the process ends at this point.

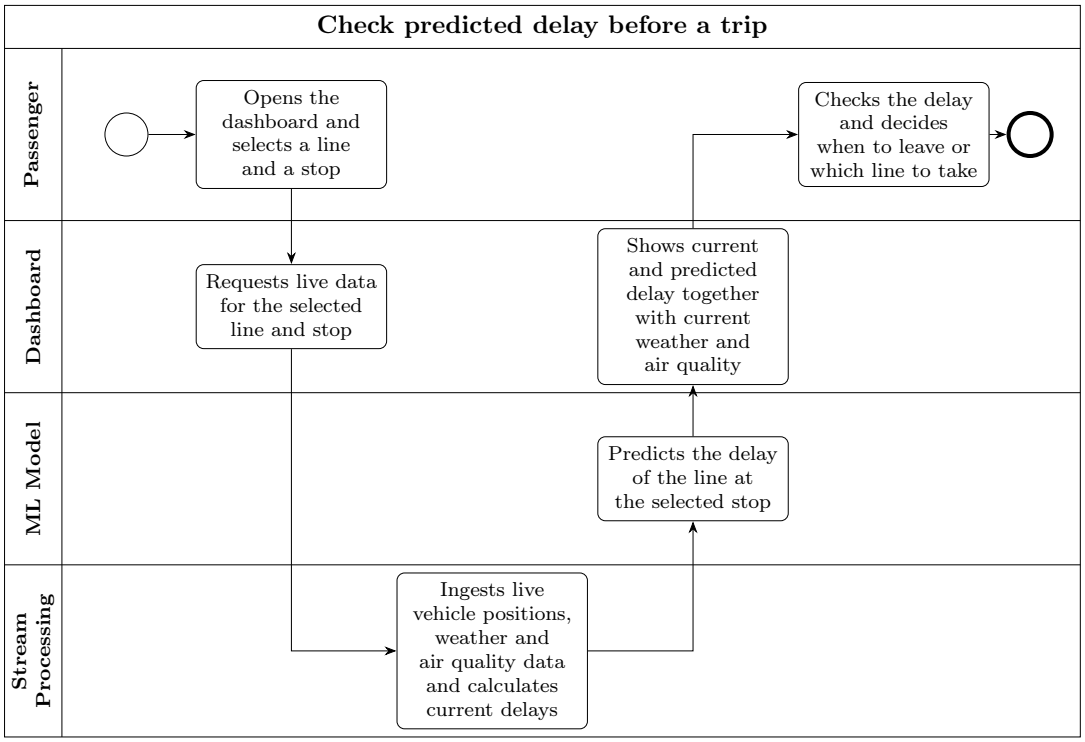


Figure 2: Business process: passenger checks the predicted delay before a trip

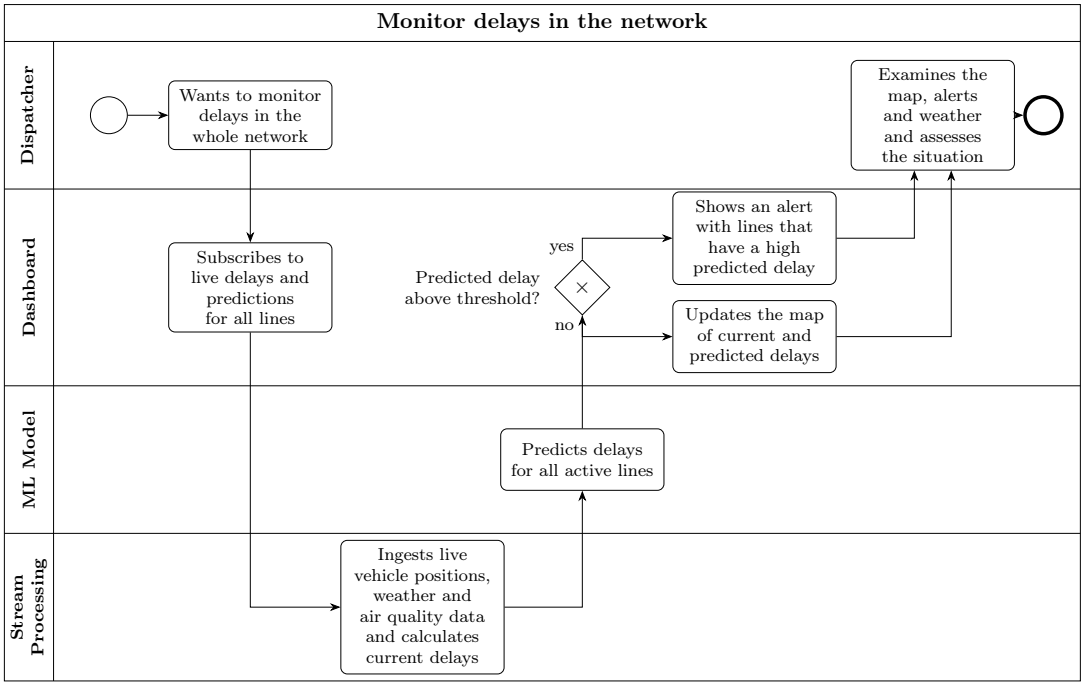


Figure 3: Business process: dispatcher monitors delays in the network

3 Datasources

Data used in the project will be obtained from data published by the city of Warsaw through its open data APIs, available at (`https://dane.um.warszawa.pl`), but with multiple endpoints of both streaming and batch nature. Polled data is returned in json format for all endpoints available. All data size estimations were acquired using models available in Gemini free tier.

3.1 Streaming data

- **Vehicle position data** - live GPS pings for every active vehicle: `Lat`, `Lon`, `Time`, `Lines`, `Brigade`, `VehicleNumber`. During rush hour there are about 450 trams and 1400 buses deployed, and such information is refreshed every 10s, though we would probably poll it in larger intervals, in order not to overload our limited infrastructure and not to exceed potential rate-limiting. A result of single poll has estimated size of 200KB.
- **Weather data** - air and surface temperature, precipitation, humidity, wind, and visibility for 18 stations located throughout the city. This data is refreshed every 10 minutes, though in order to use it as a streaming data source it is possible to poll it in shorter intervals for this purpose. A result of a single poll is estimated size of 4KB.
- **Air quality data** - returns a json with short description, monitoring station parameters, including type and location, and air quality parameters in form of nested records for 108 stations located throughout the city. This data is refreshed every 10 minutes, though in order to use it as a streaming data source it is possible to poll it in shorter intervals for this purpose. A result of a single poll is estimated size of 120KB.

3.2 Batch data

- **Timetable data** (static data, which should be acquired once per day) - stop locations, line-stop assignments, and per-brigade precise schedules for every stop-line-brigade combination in the network. This data daily (assuming daily batches) estimated size is at 140-200MB.

4 Data Processing Steps

The proposed solution is based on the Lambda Architecture, which combines the processing of current observations with the analysis of historical data. These paths support real-time delay predictions and historical statistics. Their results are brought together in a serving layer and displayed through a dashboard. Figure 4 presents the overall data flow.

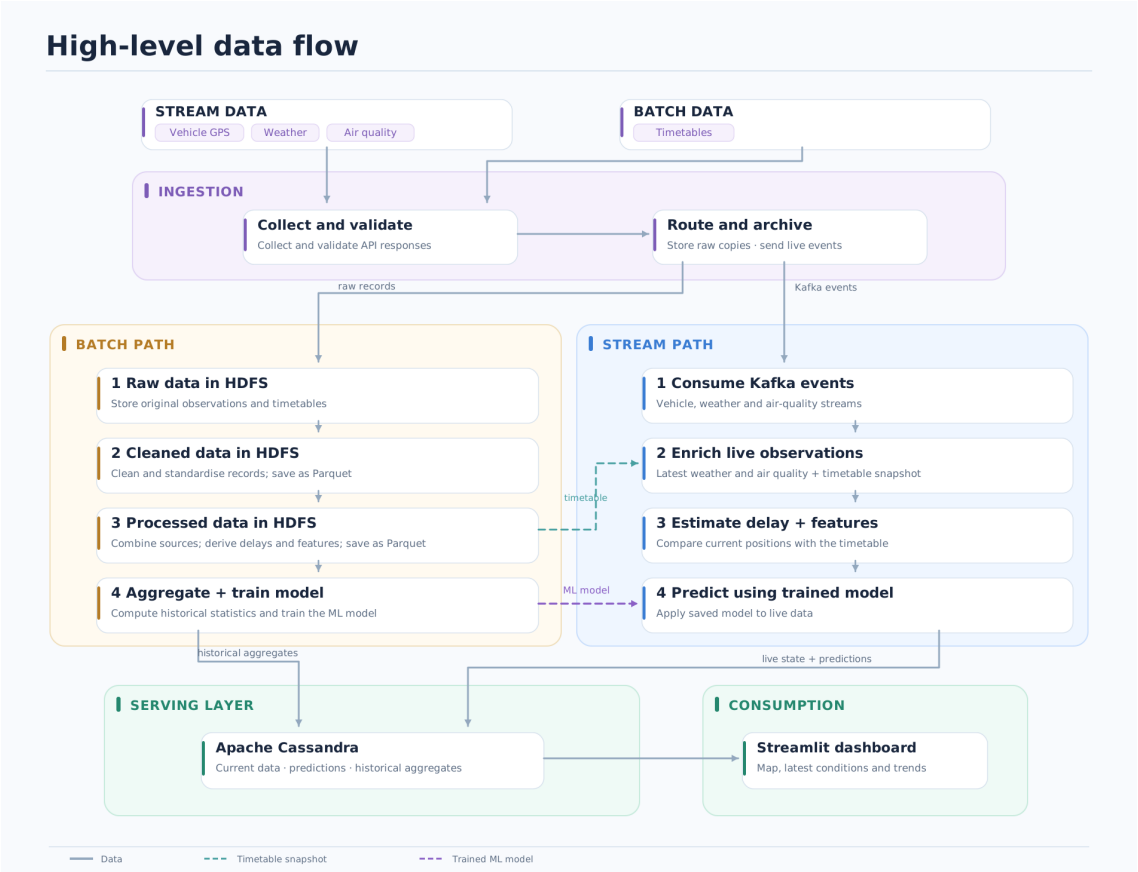


Figure 4: High-level data flow for stream and batch processing.

4.1 Data Ingestion

The system will collect three types of stream data: vehicle GPS positions, weather measurements, and air quality measurements. Public transport timetables will form a separate batch source, refreshed approximately once per day. The ingestion component will retrieve and validate data from the corresponding APIs. Original responses from all four sources will be stored in HDFS. Incoming GPS and environmental observations will also be forwarded to Kafka for stream processing.

4.2 Batch Processing

The batch path will use historical data stored in HDFS. The data will be organised into three groups. Raw will contain the original source records. Cleaned will contain validated and standardised data. Processed will contain integrated datasets, estimated delays, and features prepared for further analysis. Apache Spark will perform the cleaning and processing steps, saving both the cleaned and processed datasets in HDFS in Parquet format.

Historical GPS observations will be combined with timetables and environmental measurements to estimate delays. Spark will then calculate statistics such as average delays by line, stop, time period, and weather conditions. Historical datasets and aggregates will be retained in HDFS, while selected statistics will also be written to Cassandra for the dashboard.

A prediction model will be trained and evaluated using the processed historical data. The trained model will be stored in HDFS for use by the stream path. The batch path will also provide the latest processed timetable for real-time calculations.

4.3 Stream Processing

The stream path will receive GPS, weather, and air quality events from Kafka. Current vehicle positions will be combined with environmental measurements and the latest processed timetable. The system will estimate current delays, prepare model features, and use the model trained in the batch path to predict future delays.

Current vehicle positions, environmental measurements, estimated delays, predictions, and real-time statistics will be written to Cassandra.

4.4 Data Consumption

The Streamlit dashboard will read current information and historical aggregates from Cassandra. Users will be able to view vehicle positions, environmental conditions, historical delay patterns, and delay predictions in one application.

5 Solution Architecture

The system will use open-source Big Data technologies organised into ingestion, batch, speed, serving, and consumption components. The solution will be deployed using Docker Compose, with the main components running in containers to simplify setup and integration. Figure 5 shows the proposed technologies and their main connections.

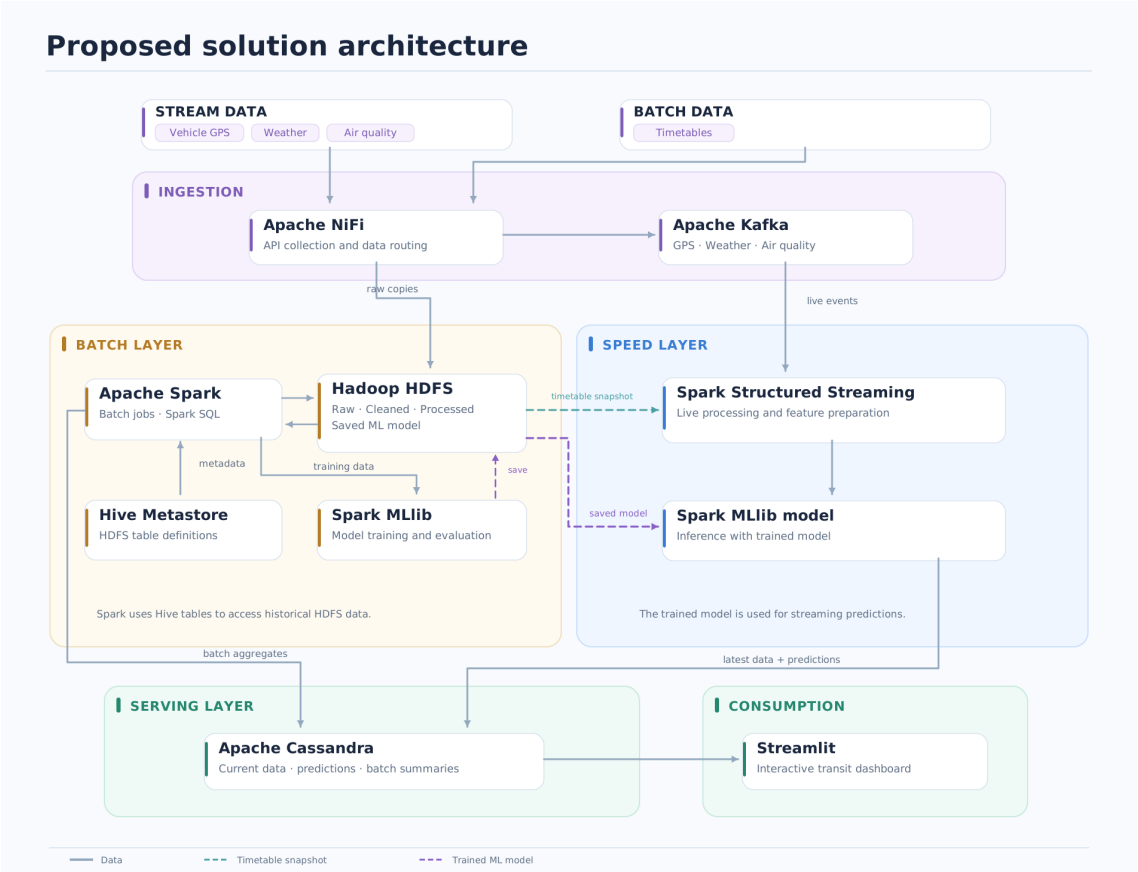


Figure 5: Proposed technology architecture based on the Lambda model.

5.1 Ingestion Layer

Apache NiFi will manage API requests and data ingestion. It will save raw observations and timetables in HDFS and send live GPS, weather, and air quality records to **Apache**

Kafka. Kafka will organise the incoming records into separate topics for the stream processing application.

5.2 Batch Layer

Hadoop HDFS will store raw, cleaned, and processed data, as well as timetable snapshots and trained models. **Apache Spark** will clean and combine historical records, prepare datasets, and calculate aggregates. **Hive Metastore** will maintain table definitions for datasets stored in HDFS, allowing them to be accessed through Spark SQL.

Spark MLlib will be used to train and evaluate the delay prediction model. The model will be saved in HDFS and made available to the speed layer.

5.3 Speed Layer

Apache Spark Structured Streaming will read events from Kafka and process current observations. It will use the latest timetable and the trained **Spark MLlib** model to calculate delays and generate predictions. Current observations, statistics, and predictions will be saved to Cassandra.

5.4 Serving Layer and Dashboard

Apache Cassandra will store current information from the speed layer and selected historical aggregates and model evaluation metrics from the batch layer. **Streamlit** will provide a dashboard that reads these results from Cassandra and presents them through interactive visualisations.

6 Use of Artificial Intelligence

AI tools will be used throughout the project to support software development, technical problem-solving, and report preparation. Their use will focus on improving productivity and assisting the team while ensuring that **all generated content and code are reviewed and verified by the team members**.

The planned uses of AI include:

- **Code development and editing:** Using AI tools integrated into the IDE to assist with writing, editing and debugging the code.
- **Technical support:** Using AI to clarify technical concepts, explain unfamiliar technologies, suggest implementation approaches, and assist in troubleshooting errors.
- **Report writing and editing:** Using AI to help structure the report, improve clarity and readability, correct grammatical and linguistic errors, and refine technical descriptions.
- **Verification of results:** Reviewing and testing AI-generated code, suggestions, and written content to ensure their correctness, relevance, and consistency with the actual project implementation.

AI will serve as a supporting tool rather than a replacement for the team's own work. All team members will remain responsible for understanding, validating, and maintaining the code and documentation they produce.

7 Division of Responsibilities

The project work is divided among four team members, each responsible for a specific component of the system and supported by another team member.

7.1 Data Ingestion and Validation

Main responsible person: Michał, **supporting team member:** Sebastian.

- Selecting and testing the Warsaw open data API endpoints for vehicle positions, weather, air quality and timetables.
- Creating scripts in order to retrieve data from external APIs.
- Configuring Apache NiFi to automate the data ingestion process, including daily retrieval of timetables.
- Storing raw copies of all collected data in HDFS (Raw layer).
- Configuring Apache Kafka topics, such as `gps`, `weather` and `air_quality`.
- Implementing data validation, error handling and duplicate detection.
- Defining and documenting data schemas.
- Documenting the data collection process.

Expected outcome: A working data ingestion pipeline that retrieves and validates data, archives raw data in HDFS and delivers live data to Apache Kafka.

7.2 Data Storage and Batch Processing

Main responsible person: Emil, **supporting team member:** Michał.

- Configuring Apache Hadoop, HDFS and Hive Metastore.
- Designing a directory structure for raw, cleaned, and processed data, timetable snapshots and trained models.
- Creating Hive Metastore table definitions for the cleaned and processed datasets and developing Spark SQL queries.
- Cleaning the collected data and handling missing or invalid values, and saving it in Parquet format.
- Estimating historical delays of buses and trams by comparing GPS positions with timetables, using delay logic shared with the stream path.
- Combining GPS, timetable, weather and air quality data into a unified dataset, including features for machine learning, with feature definitions shared with the stream path.
- Implementing batch processing jobs using Apache Spark.
- Calculating historical statistics, such as average delays of buses and trams by line, stop, hour, day of the week and weather conditions, including a ranking of weather-sensitive lines and stops.

- Training a Spark MLlib model that predicts delays of buses and trams, using the processed dataset.
- Evaluating the model with metrics and saving the trained model in HDFS for the stream path.
- Publishing the latest processed timetable snapshot in HDFS for the stream path.
- Exporting historical aggregates and model evaluation metrics to Cassandra.

Expected outcome: A functional data storage layer, a unified dataset with estimated delays and features, a repeatable batch processing pipeline for historical analysis, a trained and evaluated prediction model, and aggregates, metrics, model and timetable snapshot delivered to the serving and stream layers.

7.3 Stream Processing and Prediction

Main responsible person: Tymoteusz, **supporting team member:** Emil.

- Configuring Apache Spark Structured Streaming.
- Implementing the consumption of events from Apache Kafka.
- Enriching live observations with the latest weather and air quality data and the timetable snapshot from the batch layer.
- Calculating real-time metrics and statistics using streaming data, including current delays, with the same delay definition as in the batch path.
- Preparing features from weather, air quality and public transport data, identical to the features used for model training.
- Loading the trained model from HDFS and integrating it into the streaming pipeline to generate delay predictions for buses and trams.
- Writing current vehicle states, environmental data and predictions to Cassandra.

Expected outcome: A working stream processing pipeline capable of calculating real-time metrics and generating bus and tram delay predictions using the model trained in the batch layer.

7.4 Results Serving, Dashboard, and System Integration

Main responsible person: Sebastian, **supporting team member:** Tymoteusz.

- Configuring Apache Cassandra and creating tables for current data, predictions, historical aggregates and model metrics.
- Defining common data formats and interfaces.
- Integrating the results of batch and stream processing.
- Developing an interactive dashboard using Streamlit, reading data from Cassandra.
- Visualising a map of vehicle delays, weather and air quality conditions, bus and tram delays, historical statistics, weather impact analysis, and machine learning predictions.

- Integrating system components using Docker Compose.
- Testing the overall system and resolving integration issues.
- Preparing instructions for setting up and running the complete system.

Expected outcome: A functional dashboard presenting the results of the project, an integrated system running in Docker Compose, and documentation explaining how to run the application.